

PORTADA

Calidad de Software

Semana 10 — Pruebas unitarias, frameworks y test doubles

Universidad de Antioquía · Ingeniería de Sistemas



REPASO RELÁMPAGO

Lo que ya tenemos de la semana 9

- El proceso de pruebas no termina al ejecutar — **reportar y cerrar** también son parte de él.
- Un defecto tiene un **ciclo de vida**: Nuevo → Asignado → En progreso → Corregido → Cerrado.
- **Severidad** mide daño; **prioridad** mide urgencia — son ejes distintos.



TRANSICIÓN

Hoy: la base de la pirámide

La semana 8 cerró con la pirámide de Cohn: muchas pruebas rápidas y baratas en la base, pocas lentas y costosas en la punta. Hoy se construye esa base — las **pruebas unitarias** — y las herramientas para escribirlas de forma sistemática.



CONCEPTO

Qué es una prueba unitaria

Es una prueba automatizada que verifica el comportamiento de la **unidad más pequeña** de código con sentido propio — normalmente una función o un método — **de forma aislada** del resto del sistema.

- **Rápida:** se mide en milisegundos, no en segundos.
- **Aislada:** no depende de una base de datos, una red, ni de otras pruebas.
- **Determinista:** el mismo código produce siempre el mismo resultado.
- **Enfocada:** prueba una sola cosa, y si falla, dice exactamente qué se rompió.

CONCEPTO

Por qué "rápida y aislada" no es opcional

La pirámide de Cohn solo funciona si la base es barata: un equipo puede correr **miles** de pruebas unitarias en segundos, después de cada cambio, sin pensarlo dos veces.

Una prueba "unitaria" que se conecta a una base de datos real ya dejó de ser rápida ni aislada — es, en realidad, una prueba de integración con otro nombre. La distinción no es cosmética: cambia cuántas veces el equipo está dispuesto a ejecutarla.

CONCEPTO

Arrange-Act-Assert (AAA)

Es la estructura estándar de una prueba unitaria, en tres partes siempre en el mismo orden:

- **Arrange** (*preparar*): se crean los datos y objetos que la prueba necesita.
- **Act** (*actuar*): se llama a la función o método que se está probando.
- **Assert** (*afirmar*): se compara el resultado obtenido contra el esperado.

Una prueba que mezcla estos tres pasos sin orden es más difícil de leer cuando falla — separarlos es casi siempre la diferencia entre una prueba clara y una confusa.



CONCEPTO

Qué es una aserción

Es la instrucción que realmente **decide si la prueba pasa o falla**: compara un valor obtenido contra un valor esperado, y detiene la prueba con un mensaje claro si no coinciden.

Sin al menos una aserción, una prueba "pasa" siempre — así el código esté completamente roto. Es el error más común en quien recién empieza a escribir pruebas: ejecutar código sin afirmar nada sobre su resultado.



EJEMPLO

Una prueba unitaria completa

```
def test_calcular_descuento_cliente_premium():  
    # Arrange  
    monto = 100  
    tipo_cliente = "premium"  
  
    # Act  
    resultado = calcular_descuento(monto, tipo_cliente)  
  
    # Assert  
    assert resultado == 20
```

Tres líneas, tres responsabilidades — y si falla, el mensaje de la aserción dice exactamente qué valor se esperaba y cuál llegó.

TRANSICIÓN

Las herramientas para escribirlas

Nadie escribe pruebas unitarias a mano desde cero — existen frameworks que dan la estructura (cómo declarar una prueba), las aserciones, y el reporte de qué pasó y qué falló.



CONCEPTO · ORIGEN

JUnit, para Java

Creado por **Kent Beck** —el mismo de Extreme Programming, semana 3— y **Erich Gamma** —coautor del libro clásico de patrones de diseño "Gang of Four"— durante un vuelo en 1997. Es el origen de la familia **xUnit**, el patrón de frameworks de pruebas que después se replicó en casi todos los lenguajes.

Una prueba en JUnit se marca con la anotación `@Test`, y usa métodos como `assertEquals()` para las aserciones.



CONCEPTO

pytest, para Python

A diferencia de JUnit, **pytest** no exige heredar de una clase especial ni usar anotaciones: cualquier función cuyo nombre empiece con `test_` ya es una prueba, y la palabra reservada `assert` de Python —la misma que se usa en cualquier script— es la aserción.

Esa simplicidad es su principal argumento de venta frente a `unittest`, el framework que viene incluido en Python pero que sí exige la estructura de clases al estilo xUnit.



TRANSICIÓN

Qué pasa cuando la unidad depende de algo más

La regla era "aislada" — pero casi ninguna función real vive sola. Llama a una base de datos, a una API externa, o envía un correo. Probarla de verdad contra esas dependencias reales rompería la velocidad y el aislamiento. Hace falta una forma de "fingir" esas dependencias.



CONCEPTO · ORIGEN

Qué es un test double

El término lo acuñó **Gerard Meszaros** en su libro "*xUnit Test Patterns*" (2007), tomando la metáfora del cine: un **doble de riesgo** reemplaza al actor real en una escena peligrosa — se parece lo suficiente para la escena, sin ser el actor de verdad.

Un **test double** es, igual, un objeto que reemplaza una dependencia real dentro de una prueba, imitando lo necesario para que la prueba funcione — sin la complejidad ni la lentitud de la dependencia real. Stub, mock y fake son los tres tipos más comunes.

CONCEPTO

Stub

Es un test double que **siempre devuelve una respuesta predefinida**, sin importar cómo se le llame. No verifica nada sobre cómo fue usado — solo está ahí para que el código bajo prueba tenga algo que recibir.

Ejemplo: un stub de "servicio de clima" que siempre devuelve `"soleado"`, para probar una función que decide si mostrar un mensaje de alerta según el clima — sin depender de un servicio meteorológico real.



CONCEPTO

Mock

Es un test double que, además de devolver respuestas predefinidas, **verifica que se le llamó de la forma esperada** — cuántas veces, con qué argumentos, en qué orden. Si la interacción esperada no ocurrió, el mock hace fallar la prueba.

Ejemplo: un mock del "servicio de correo" que confirma que `enviar_correo()` se llamó exactamente una vez, con el destinatario correcto, al procesar un pedido — sin enviar un correo real.



CONCEPTO

Fake

Es un test double con una **implementación realmente funcional, pero simplificada** — no solo devuelve datos fijos, de verdad hace el trabajo, solo que de forma más liviana que la dependencia original.

Ejemplo: una base de datos en memoria que implementa guardar y consultar de verdad, en vez de conectarse a un servidor de base de datos real — se comporta como una base de datos, pero sin la infraestructura.



CONCEPTO · SÍNTESIS

Stub, mock y fake, lado a lado

Tipo	Qué hace	Verifica interacción
Stub	Devuelve respuestas predefinidas	No
Mock	Devuelve respuestas y registra cómo se le llamó	Sí
Fake	Implementación real simplificada	No

La pregunta para elegir cuál usar: ¿me importa qué devuelve (stub), cómo se le llamó (mock), o necesito que realmente funcione (fake)?

EJEMPLO

Una prueba con un mock

```
def test_procesar_pedido_envia_correo():  
    # Arrange  
    servicio_correo = Mock()  
    pedido = Pedido(cliente="ana@example.com", total=50)  
  
    # Act  
    procesar_pedido(pedido, servicio_correo)  
  
    # Assert  
    servicio_correo.enviar.assert_called_once_with(  
        destinatario="ana@example.com"  
    )
```

La aserción no compara un valor de retorno — verifica que `enviar()` se haya llamado exactamente una vez, con el destinatario correcto.

INTERACCIÓN

Ejercicio rápido: ¿stub, mock o fake?

5 minutos — clasifiquen cada caso:

1. Un objeto que siempre devuelve `true` al preguntar "¿el usuario tiene permiso?"
2. Un objeto que confirma que se llamó `guardar_log()` exactamente tres veces.
3. Una lista en memoria que reemplaza una tabla de base de datos, y de verdad permite agregar y buscar elementos.

Este ejercicio es el calentamiento del taller de hoy.

CONCEPTO EN DISPUTA

¿Muchos mocks son señal de mal diseño?

Una prueba que necesita cinco test doubles para aislar una sola función suele ser una señal —no del test, sino del código: esa función probablemente tiene demasiadas responsabilidades o dependencias mezcladas.

La postura razonable: los test doubles son una herramienta legítima y necesaria, pero cuando su cantidad crece sin control, **el problema real casi siempre está en el diseño del código**, no en la prueba — vale la pena refactorizar (semana 5) antes de agregar el mock número seis.



SÍNTESIS

Lo que hay que llevarse de hoy

1. Una prueba unitaria es **rápida, aislada, determinista y enfocada** — es la base de la pirámide de Cohn.
2. **Arrange-Act-Assert** ordena cualquier prueba en tres pasos claros.
3. **JUnit** (Java) y **pytest** (Python) son frameworks de la familia xUnit, con distinto nivel de formalidad.
4. Un **test double** reemplaza una dependencia real — **stub** devuelve datos fijos, **mock** verifica interacción, **fake** funciona de verdad, simplificado.
5. Demasiados mocks en una sola prueba suelen señalar un problema de diseño, no de la prueba.



INSTRUCCIONES

Taller de hoy: suite de pruebas unitarias

2 horas, en sala de Zoom asignada — equipo fijo de Fábrica-Escuela

El docente entrega una **regla de negocio** para implementar. Cada equipo debe escribir una suite de pruebas unitarias que incluya:

- Casos derivados de **particiones de equivalencia y valores límite** (semana 8) sobre las entradas de la regla.
- Al menos **un test double** (stub, mock o fake) para alguna dependencia externa de la regla.
- Estructura **Arrange-Act-Assert** en cada prueba.

La entrega cierra al terminar las dos horas. Recuerden: al enviar, cada integrante reporta en privado quién trabajó.



CIERRE

Para la próxima semana

Antes de la clase de la semana 11, hay que llegar con esto leído:

- Métricas de calidad de software: densidad de defectos, DRE, cobertura, complejidad ciclomática.

Seguimos con los equipos fijos de **Fábrica-Escuela**. El taller de la semana 11: cálculo de métricas sobre un repositorio asignado, con interpretación escrita de qué revela y qué oculta cada una.

